

Functions

Overview

A function is a reusable block of code that performs a task.

Functions help:

- organize code
 - avoid repetition
 - make programs easier to read and debug
-

Topic: Function Basics

Key Vocabulary

- Function definition
 - The function's name plus the block of statements inside it
 - Function call
 - Running the function by using its name
 - Return statement
 - Sends a value back from the function
 - Parameter
 - An input listed in the function definition
 - Argument
 - The actual value passed into the function when it is called
-

Creating a Function

Use the `def` keyword to define a function.

```
def calc_pizza_area():  
    pass
```

Example with Parameters

```
def calc_pizza_area(pizza_diameter):  
    return pizza_diameter * pizza_diameter
```

```
def calc_pizza_area(pizza_diameter, pizza_height):  
    return pizza_diameter * pizza_height
```

Function Calls

```
calc_pizza_area()  
calc_pizza_area(12.0)  
calc_pizza_area(12.0, 13.0)
```

Return Statements

Notes

- A function can return only one value directly
- A function can still return multiple pieces of data by packaging them in a tuple or list
- If no `return` statement is used, Python returns `None`
- A function can have multiple `return` statements
- A `return` statement can appear anywhere inside the function

Example

```
def multiply(size, length):  
    return size * length
```

Topic: Printing from a Function

Notes

- A function that only prints usually does not return a value
- A function with no `return` statement is sometimes called a void function
- In Python, a function with no return value still returns `None`

Example

```
def greet(name):  
    print(f"Hello, {name}")
```

Topic: Dynamic Typing

Notes

Python uses dynamic typing, which means object types are determined while the program runs.

This means the same function can work with different data types, depending on the operation.

Examples

```
def add(a, b):  
    return a + b
```

```
add(5, 7)
```

Output:

```
12
```

```
add("Tora", "Bora")
```

Output:

```
ToraBora
```

Polymorphism

Polymorphism means the same operation can behave differently depending on the type.

Example

```
"x" * 5
```

Output:

```
xxxxx
```

Topic: Function Stubs

Definition

A function stub is a function definition whose full code has not been written yet. This is useful when planning a program before finishing every function.

Using `pass`

The `pass` keyword does nothing, but it works as a placeholder.

```
def calc_volume(radius):  
    pass
```

Using a Temporary Message

A stub can print a reminder for the programmer and return a placeholder value.

```
def steps_to_calories(steps):  
    print("FIXME: finish function")  
    return -1
```

Notes

- Returning `1` is a common placeholder for unfinished functions that should normally return a value

Using `NotImplementedError`

You can stop the program and clearly show the function is unfinished.

```
def get_points(num_points):  
    raise NotImplementedError
```

Notes

- `raise` is used to generate an error manually
- `NotImplementedError` shows the function is not finished yet

Topic: Scope of Variables and Functions

Definition

Scope is the part of a program where a variable or function is visible and usable.

Local Variables

A local variable is created inside a function.

Notes

- It only exists inside that function
 - Other parts of the program cannot access it directly
-

Global Variables

A global variable is created outside functions.

Notes

- It can be accessed throughout the file
- A function can read a global variable
- To change a global variable inside a function, use `global`

Example

```
student_name = "Armando"

def get_name():
    global student_name
    student_name = "Mando"
```

Mutable Global Variables

If the global variable is mutable, like a list or dictionary, you usually do not need `global` just to modify its contents.

Example idea

- adding to a list
- removing from a dictionary
- updating a key-value pair

This works because the object itself is being changed, not reassigned

Topic: Function Arguments

How Python Passes Arguments

Python uses pass-by-object-reference.

That means:

- the parameter becomes a new local name
- both the argument and parameter refer to the same object

Whether changes affect the original value depends on mutability.

Immutable Objects

Examples:

- `int`
- `float`
- `str`
- `tuple`

Notes

- Immutable objects cannot be changed in place
- Reassigning the parameter creates a new object
- The caller's original value stays the same

Example idea

```
def change_num(x):  
    x = 10
```

This does not change the original outside value.

Mutable Objects

Examples:

- `list`
- `dict`
- `set`

Notes

- Mutable objects can be changed in place
 - Changes inside the function can appear outside the function too
 - Common examples:
 - `list.append()`
 - `list.sort()`
 - changing `dict[key]`
-

Protecting Mutable Arguments

If you do not want the original object changed, pass a copy.

Example

```
my_func(my_list[:])
```

Notes

- `my_list[:]` creates a shallow copy of the list
-

Topic: Keyword Arguments

Definition

Keyword arguments let you pass values by parameter name instead of position.

Function Definition

```
def student_info(name, age, grade, gpa):  
    print(name, age, grade, gpa)
```

Function Call with Keywords

```
student_info(age=28, name="Armando", gpa=4.0, grade="A")
```

Notes

- Order does not matter when using keyword arguments
 - Parameter names must match exactly
-

Default Parameter Values

A parameter can have a default value.

That means:

- the argument becomes optional
- if no value is passed, Python uses the default

Example

```
def print_date(day=1, month=1, year=2000, style=0):  
    print(day, month, year, style)
```

Valid Function Calls

```
print_date(30, 7, 2012, 0)    # no defaults used
print_date(30, 7, 2012)       # style = 0
print_date(30, 7)             # year = 2000, style = 0
print_date(30)                # month = 1, year = 2000, style
= 0
print_date()                  # day = 1, month = 1, year = 200
0, style = 0
```

Important

If a parameter has no default value, leaving it out causes an error.

Topic: Arbitrary Arguments

Why They Are Used

Sometimes a programmer does not know how many arguments a function will need.

Python supports:

- `args` for extra positional arguments
 - `*kwargs` for extra keyword arguments
-

args

Definition

- `args` collects extra positional arguments into a tuple.

Example

```
def print_student(name, age, *args):
    print(f"{name} is {age}", end=" ")
```

```
if len(args) > 0:
    print("and is graduating in", end=" ")

for extra in args:
    print(extra, end=" ")

print("")
```

Function Calls

```
print_student("Armando", 28)
print_student("Armando", 28, 2027)
```

Output

```
Armando is 28
Armando is 28 and is graduating in 2027
```

*kwargs

Definition

- `*kwargs` collects extra keyword arguments into a dictionary.

Example

```
def print_student(name, age, **kwargs):
    print(f"{name} is {age}")

    for category, extra in kwargs.items():
        print(f"    {category}: {extra}")

    print()
```

Function Call

```
print_student("Armando", 28, Graduation="2027")
```

Output

```
Armando is 28  
  Graduation: 2027
```

Extra Notes About `args` and `*kwargs`

- The `*` symbols matter
- `args` and `kwargs` are common names, but other valid identifiers could be used
- One or both can be used in the same function
- They must come last in the parameter list
- If both are used, the order should be:
 - regular parameters
 - `args`
 - `*kwargs`

Topic: Multiple Function Outputs

Problem

A function can return only one value directly.

Solution

Package multiple outputs into one container, usually a tuple, and return that tuple.

Example

```
student_scores = [75, 84, 66, 99, 51, 65]

def get_grade_stats(scores):
    mean = sum(scores) / len(scores)

    tmp = 0
    for score in scores:
        tmp += (score - mean) ** 2

    std_dev = (tmp / len(scores)) ** 0.5

    return mean, std_dev
```

Unpacking the Returned Tuple

```
average, standard_deviation = get_grade_stats(student_scores)

print(f"Average score: {average}")
print(f"Standard deviation: {standard_deviation}")
```

Output

```
Average score: 73.33333333
Standard deviation: 15.260697
```

Notes

- Parentheses are optional when returning a tuple

These two are equivalent:

```
return mean, std_dev
```

```
return (mean, std_dev)
```

- You could also return a list instead

```
return [mean, std_dev]
```

Unpacking

Unpacking means assigning multiple values from a tuple or list into separate variables.

Example

```
average, standard_deviation = get_grade_stats(student_scores)
```

This assigns:

- `average` to the first returned value
- `standard_deviation` to the second returned value

Quick Summary

Main ideas

- `def` creates a function
- parameters are inputs in the definition
- arguments are values passed during the call
- `return` sends a value back
- functions with no `return` give back `None`

Important concepts

- local vs global scope

- mutable vs immutable arguments
- keyword arguments
- default parameter values
- `args` and `*kwargs`
- returning multiple outputs using a tuple